ARTIFICIAL INTELLIGENCE LAB PRACTICAL

Design and Implementation of a Simple Reflex Agent in the Vacuum-Cleaner World

1. AIM

To design, implement, and analyze a Simple Reflex Agent for a two-state Vacuum-Cleaner World, and to evaluate its rational behavior under varying environmental conditions.

2. OBJECTIVES

- Understand the core concepts of the PEAS (Performance, Environment, Actuators, Sensors) framework.
- Implement a condition-action rule mapping system for an autonomous agent.
- Analyze what makes an agent 'rational' or 'irrational' based on performance metrics and environment visibility.

3. SYSTEM REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 / AMD Ryzen 3 or higher
- Memory (RAM): 4 GB minimum (8 GB recommended)
- Storage: 20 GB of free hard disk space

Software Requirements:

- Operating System: Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
- Programming Language: Python 3.8 or higher
- Development Environment: IDLE, VS Code, PyCharm, or Jupyter Notebook

4. THEORY & PEAS DESCRIPTION

A Simple Reflex Agent is the most basic type of AI agent. It selects actions based exclusively on the current percept, completely ignoring the history of past percepts. It operates on strict condition-action rules (e.g., if status == Dirty then Suck).

To formally evaluate the agent's behavior, we define its operational boundary using the PEAS framework:

- **Performance Measure:** +10 points for each clean square per time step; -1 point for each moving or sucking action taken.
- **Environment:** A simple 2-state world comprising Location A and Location B. Each location can be independently 'Clean' or 'Dirty'.
- **Actuators:** Wheels/Motors (to move 'Left' or 'Right'), Vacuum Pump (to 'Suck' dirt), and 'NoOp' (No Operation / Turn off).
- **Sensors:** Location sensor (identifies if the agent is in A or B) and Dirt sensor (identifies if the current square is 'Clean' or 'Dirty').

5. IMPLEMENTATION STEPS

Step 1: Define the Environment State
Create an Environment class that tracks the agent's current location ('A' or 'B') and the status of both rooms ('Clean' or 'Dirty').

Step 2: Build the Simple Reflex Agent Logic
Implement the rule-base loop: If current location is dirty, return 'Suck'. If in location A and clean, return 'Right'. If in location B and clean, return 'Left'.

Step 3: Simulate the Step-by-Step Execution
Create a running loop that allows the agent to perceive the environment, take action, mutate the environment's state, and record performance penalties or rewards over multiple discrete time intervals.

6. PYTHON CODE IMPLEMENTATION

Below is the complete, self-contained Python program representing the agent and its simulation environment:

```
import random

class VacuumEnvironment:
    def __init__(self):
        # Randomly initialize environment state
        self.locations = {
            'A': random.choice(['Clean', 'Dirty']),
            'B': random.choice(['Clean', 'Dirty'])
        }
        self.agent_location = random.choice(['A', 'B'])

    def get_percept(self):
        # Returns current location and its status to the agent
        return (self.agent_location, self.locations[self.agent_location])

    def execute_action(self, action):
```

```
        cost = 0
        if action == 'Suck':
            self.locations[self.agent_location] = 'Clean'
            cost = 1  # Action cost
            print(f"Action: Sucked dirt at Location {self.agent_location}.")
        elif action == 'Right' and self.agent_location == 'A':
            self.agent_location = 'B'
            cost = 1
            print("Action: Moved Right to Location B.")
        elif action == 'Left' and self.agent_location == 'B':
            self.agent_location = 'A'
            cost = 1
            print("Action: Moved Left to Location A.")
        elif action == 'NoOp':
            print(f"Action: No Operation at Location {self.agent_location}.")
        return cost

class SimpleReflexAgent:
    def __init__(self):
        pass

    def interpret_input(self, percept):
        # Condition-Action Rules mapping
        location, status = percept
        if status == 'Dirty':
            return 'Suck'
        elif location == 'A':
            return 'Right'
        elif location == 'B':
            return 'Left'
        return 'NoOp'

# --- Simulation Execution ---
if __name__ == "__main__":
    env = VacuumEnvironment()
    agent = SimpleReflexAgent()

    print("--- Initial Environment State ---")
    print(f"Agent starts at: {env.agent_location}")
    print(f"Room States: A={env.locations['A']}, B={env.locations['B']}\n")

    total_performance = 0
    steps = 5  # Run for 5 discrete time intervals

    for step in range(1, steps + 1):
        print(f"--- Time Step {step} ---")
        percept = env.get_percept()
        print(f"Sensor Percept: Location {percept[0]} is {percept[1]}")

        # Agent decides action
```

```
        action = agent.interpret_input(percept)

        # Environment processes action
        action_cost = env.execute_action(action)

        # Performance calculation for this step
        step_reward = 0
        if env.locations['A'] == 'Clean': step_reward += 10
        if env.locations['B'] == 'Clean': step_reward += 10

        step_score = step_reward - action_cost
        total_performance += step_score

        print(f"Current State: A={env.locations['A']}, B={env.locations['B']}")
        print(f"Step Score: {step_score} | Accumulative Score: {total_performance}\n")

    print(f"Simulation ended. Final Agent Score: {total_performance}")
```

7. CONCLUSION

In this practical, a Simple Reflex Agent for the Vacuum-Cleaner world was designed and successfully implemented in Python. The experiment demonstrates that while Simple Reflex Agents are computationally efficient and highly effective in fully observable, static environments, their lack of historical memory limits their efficiency when facing unobserved states, stochastic environment transitions, or dynamic changes.